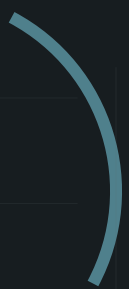


Intelligent Control of Robots with Minimal Power Consumption in Pick-and-Place Operations





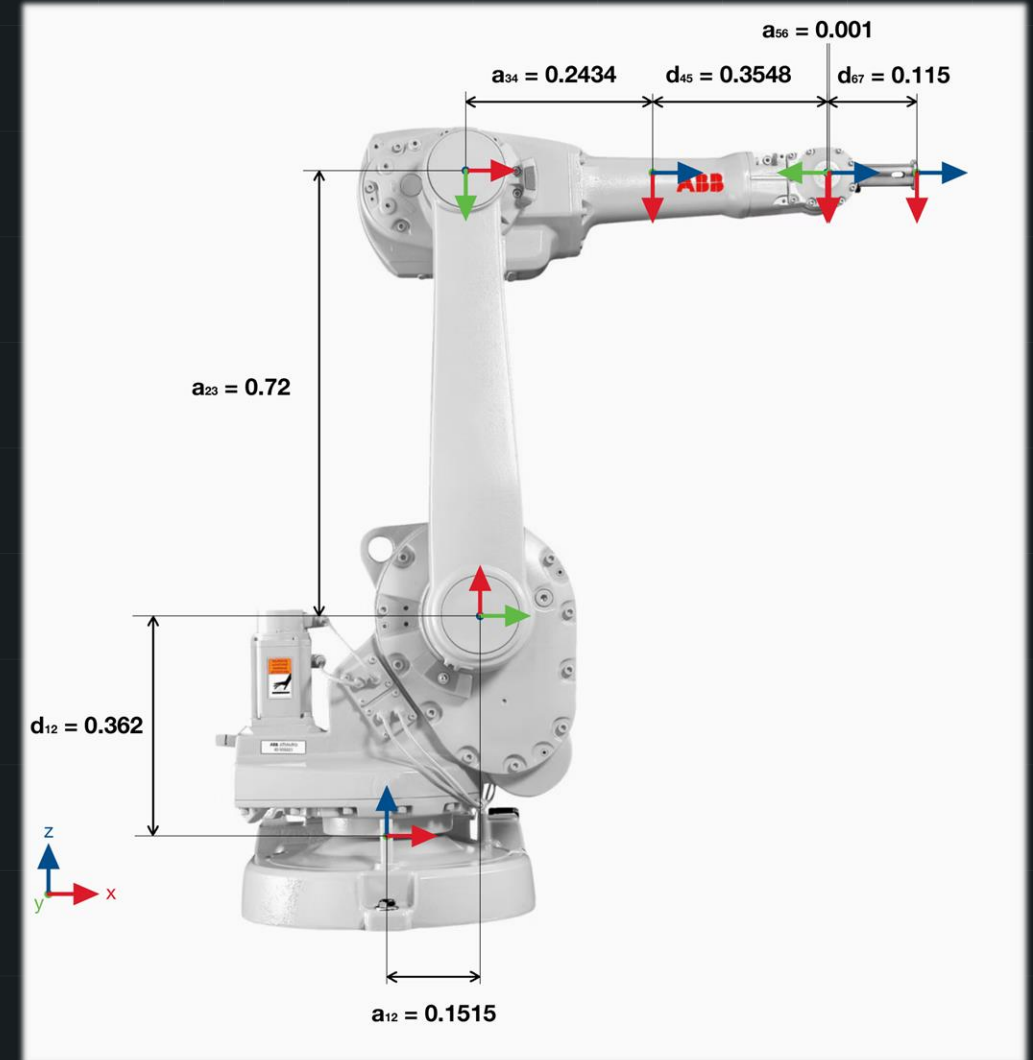
Rigid Body Tree

Robot Model & Visualization

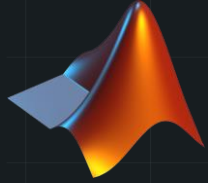
DH Table adjustments and RBT's construction

- Several changes have been performed to the DH table's parameters of the original paper.
- Precisely speaking, only a and d parameters have been changed to their realistic values.

Body	DH Parameters			
	a	α	d	θ
B1	a_{12}	$-\pi/2$	d_{12}	θ_1
B2	a_{23}	0	0	$\theta_2 - \pi/2$
B3	a_{34}	0	0	$\theta_3 + \pi/2$
B4a	0	$\pi/2$	0	$\pi/2$
B4	0	$-\pi/2$	d_{45}	θ_4
B5	a_{56}	0	0	$\theta_5 - \pi/2$
B6a	0	$\pi/2$	0	$\pi/2$
B6	0	0	d_{67}	θ_6



A first implementation on MATLAB...



- A first approach has been the creation of the links and the joints on MATLAB code.

- All the bodies have been set with their own CoMs, Inertias and constraints of their joints.



- Then, each one has been connected to its proper mesh.



- Finally, some fixed transformations to translate and rotate the bodies have been applied.

Centro di massa 0.051 m, -0.009 m, 0.214 m

▼ Momento d'inerzia al centro di massa (kg m²)

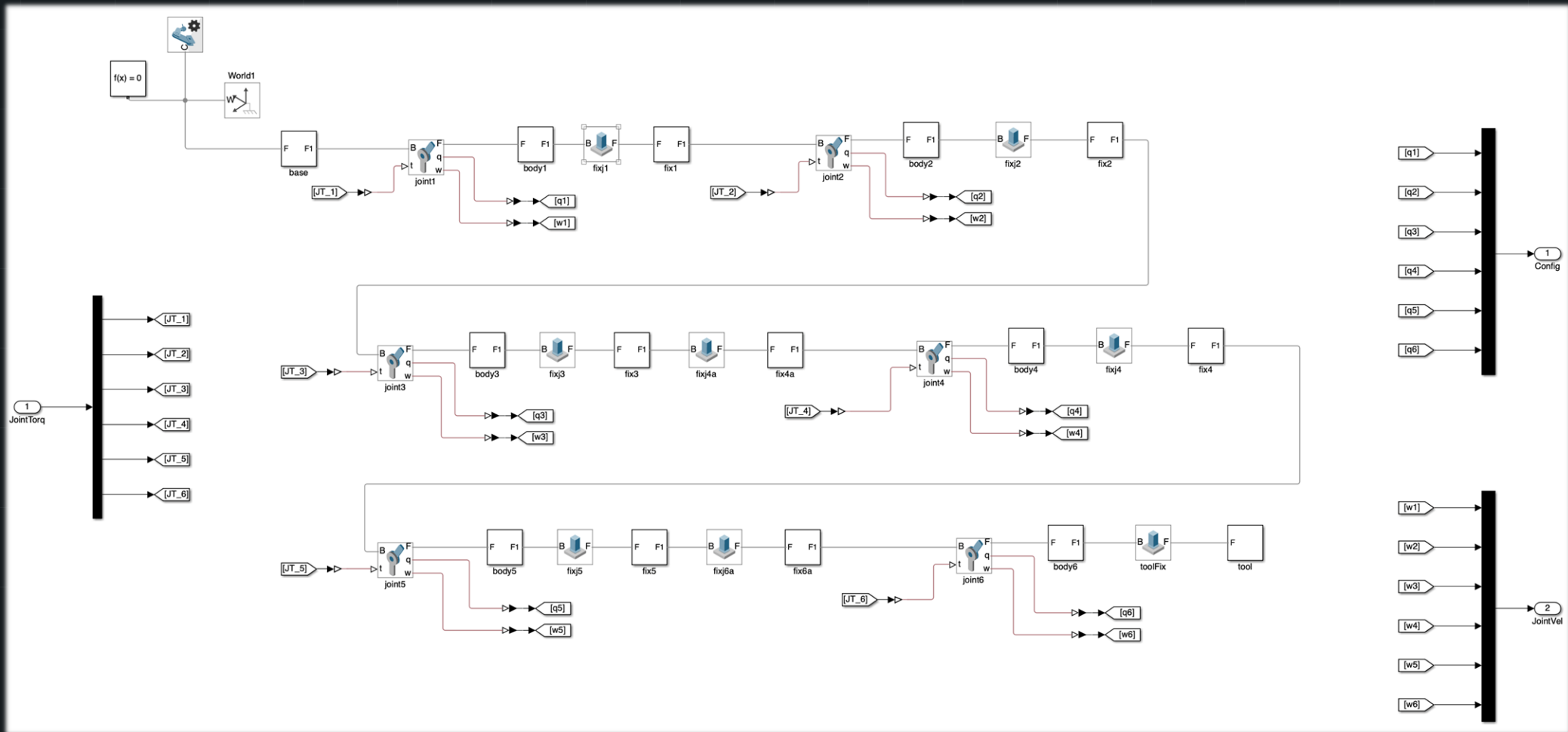
Ixx = 1.479	Ixy = 0.11	Ixz = -0.466
Iyx = 0.11	Iyy = 2.435	Iyz = 0.157
Izx = -0.466	Izy = 0.157	Izz = 1.744



```
%% ----- 1. JOINT 1 (Turret) -----  
body1 = rigidBody("body1");  
body1.Mass = 70.568;  
body1.CenterOfMass = [0.051, -0.009, 0.214];  
body1.Inertia = [1.479, 2.435, 1.744, 0.157, -0.466, 0.11];  
  
T_mesh1 = S;  
addVisual(body1, "Mesh", "link_1_turret.stl", T_mesh1);
```

... to pass to Simscape Multibody

- The RBT has been firstly exported to Simscape Multibody via **smimport()** and then hierarchically restructured.

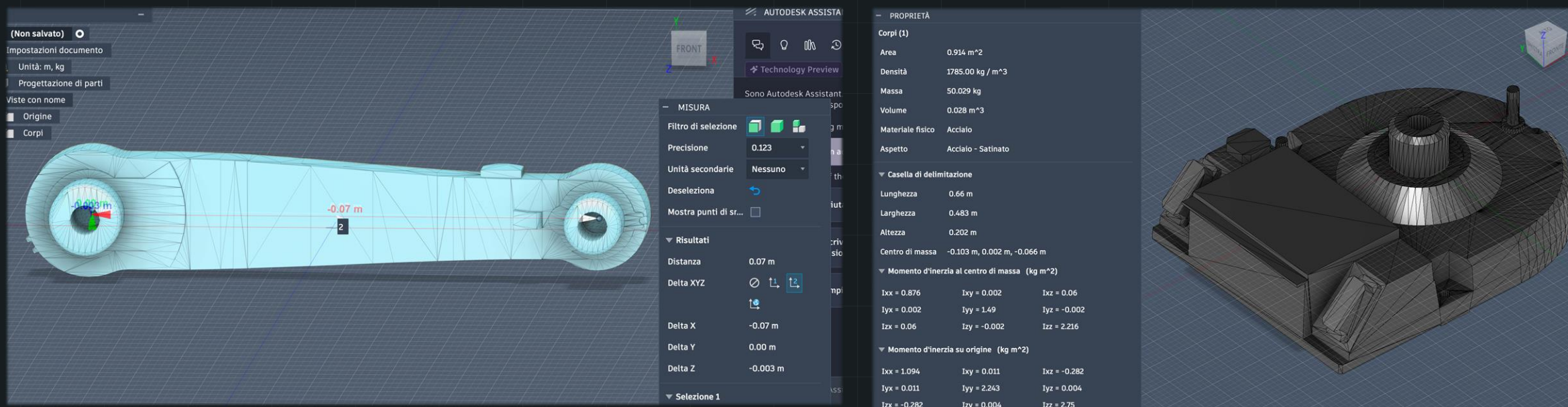
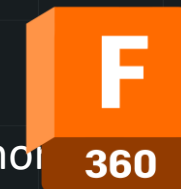


Visualization

Attaching the meshes to the Rigid
Body Tree

Autodesk elaboration

- To align the IRB1600's meshes to their respective reference frames, these .stl files have been translated and rotated on Autodesk Fusion.
- Several measurements have been performed to derive the DH Table parameters's real values. Furthermore the simulated CoMs and Inertias were extrapolated.



FK/IK

Inverse and forward kinematics

Inverse and forward kinematics

CASE STUDY

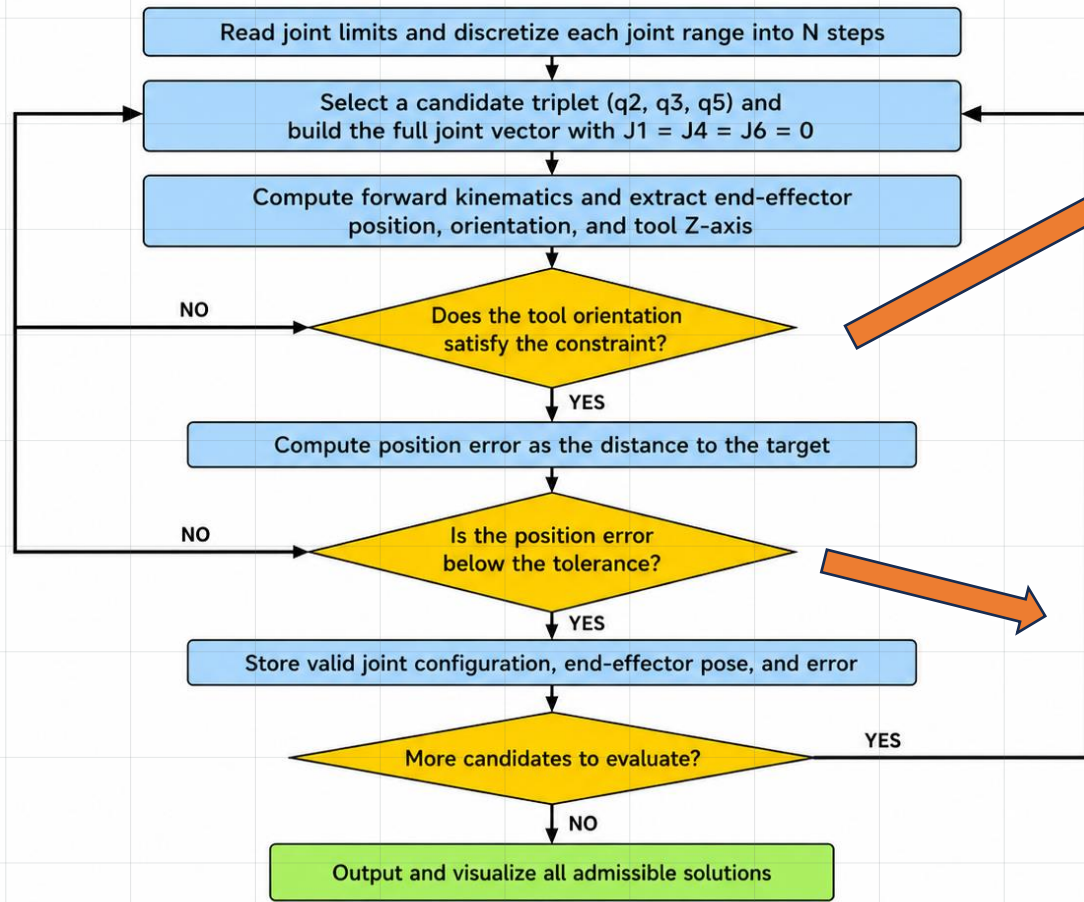
- The Pick and Place problem has been simplified to a 2D task
- Joints 1 4 and 6 are locked to zero
- Intrinsically reduce power consumption
- This cause the end effector to move only on XZ plane
- The task is expressed in 2 independet Carthesian variables (x,z)variables while
- Moovable joint are 3 so the problem is redundant
- Initially an inverse kinematic approach was considered
- In the end a forward kinematic based algorithm was impelmented to solve the inverse kinematic problem

```
q =  
    0    1    1    0    1    0  
  
>> J = geometricJacobian(robot, q, "tool")  
  
J =  
  
-0.0000 -0.0000 -0.0000 -0.4161 -0.0000 -0.9900  
-0.0000  1.0000  1.0000  0.0000  1.0000  0.0000  
 1.0000  0.0000  0.0000 -0.9093  0.0000 -0.1411  
-0.0000 -0.1713 -0.5603 -0.0000 -0.0164  0.0000  
 0.3936  0.0000 -0.0000  0.0976 -0.0000  0.0000  
 0.0000 -0.2421  0.3638  0.0000  0.1148 -0.0000
```

Inverse and forward kinematics

Inverse Kinematics via Forward Kinematics Search

Exhaustive search in the reduced joint space (J_2, J_3, J_5)



```
if zTool(3) < -1 || zTool(3) > -0.9
    continue
end
```

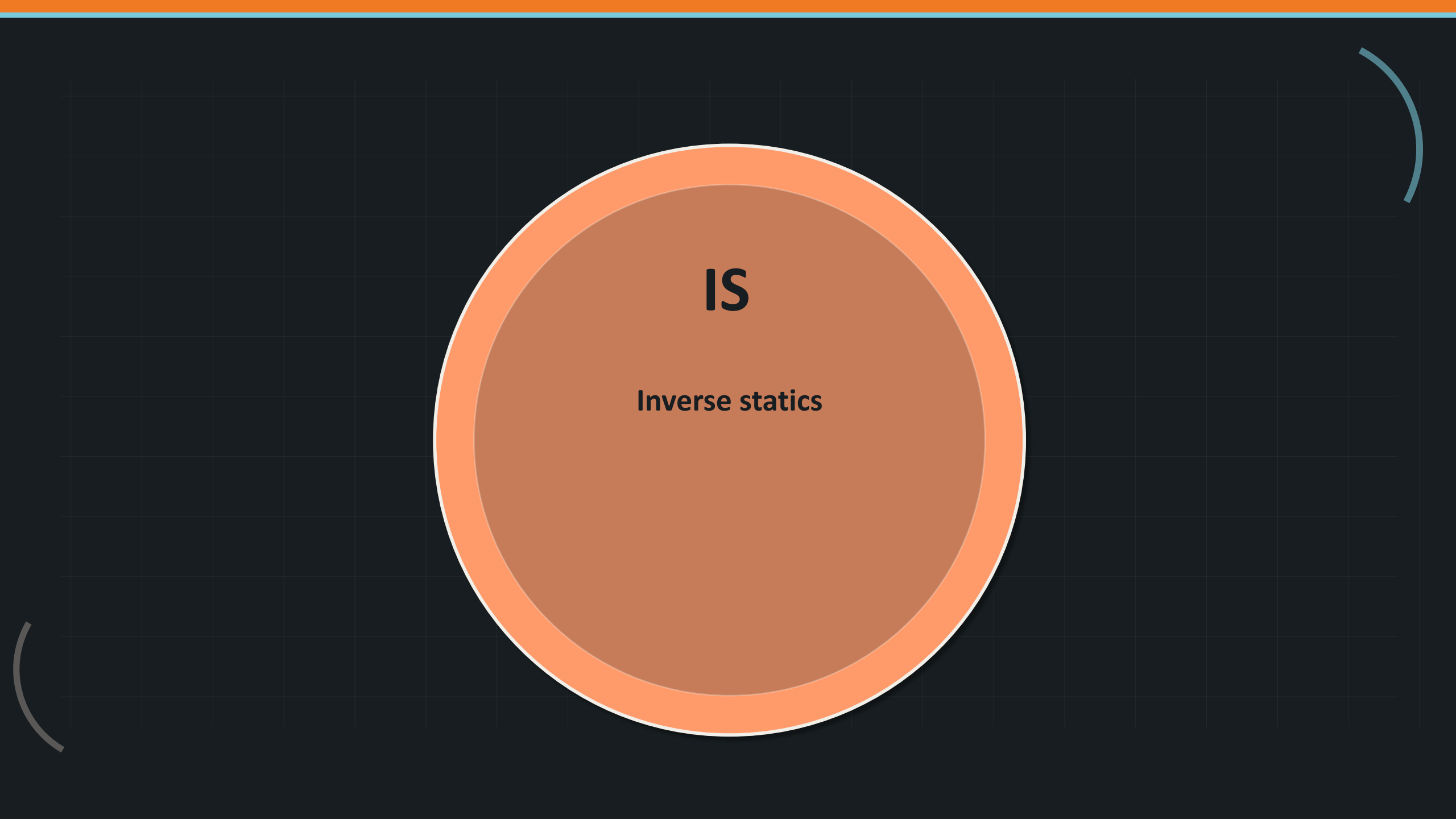
```
err = norm(pos1 - targetPos);
if err < tol
    errors = [errors; err];
    EE_pose = [EE_pose; ee_pose]; % Store the end effector pose
    Q_final = [Q_final; q]; % Store the joint angles for visualization
end
```

Inverse and forward kinematics

Ten feasible joint configurations were obtained for the same target pose, providing multiple candidate solutions for the inverse static comparison.

Joint configurations

Configuration	Joint1	Joint2	Joint3	Joint4	Joint5	Joint6
1	0	0.1812	0.4499	0	0.5859	0
2	0	0.1812	0.4499	0	0.6263	0
3	0	0.1812	0.4499	0	0.6667	0
4	0	0.2117	0.3970	0	0.8687	0
5	0	0.2117	0.3970	0	0.9091	0
6	0	0.2117	0.3970	0	0.9495	0
7	0	0.2727	0.3440	0	1.1919	0
8	0	0.2727	0.3440	0	1.2323	0
9	0	0.2727	0.3440	0	1.2727	0
10	0	0.2727	0.3440	0	1.3131	0



IS

Inverse statics

Inverse statics

Inverse statics was used to estimate the joint torques required to keep each feasible robot configuration in equilibrium under the applied end-effector force.

$$\tau_j = \tau_{jf} + Kq + B\dot{q} + J\ddot{q}$$

At the given supply voltage U :

$$P_j = UI = (U * \tau_j) / \psi$$

$$E_j = P_j * t$$

- τ_j = total joint torque
- τ_{jf} = gravity torque
- Kq = elastic/stiffness contribution
- $B\dot{q}$ = damping/viscous contribution
- $J\ddot{q}$ = inertial contribution

Energy proportional to the total joint torque

Inverse statics

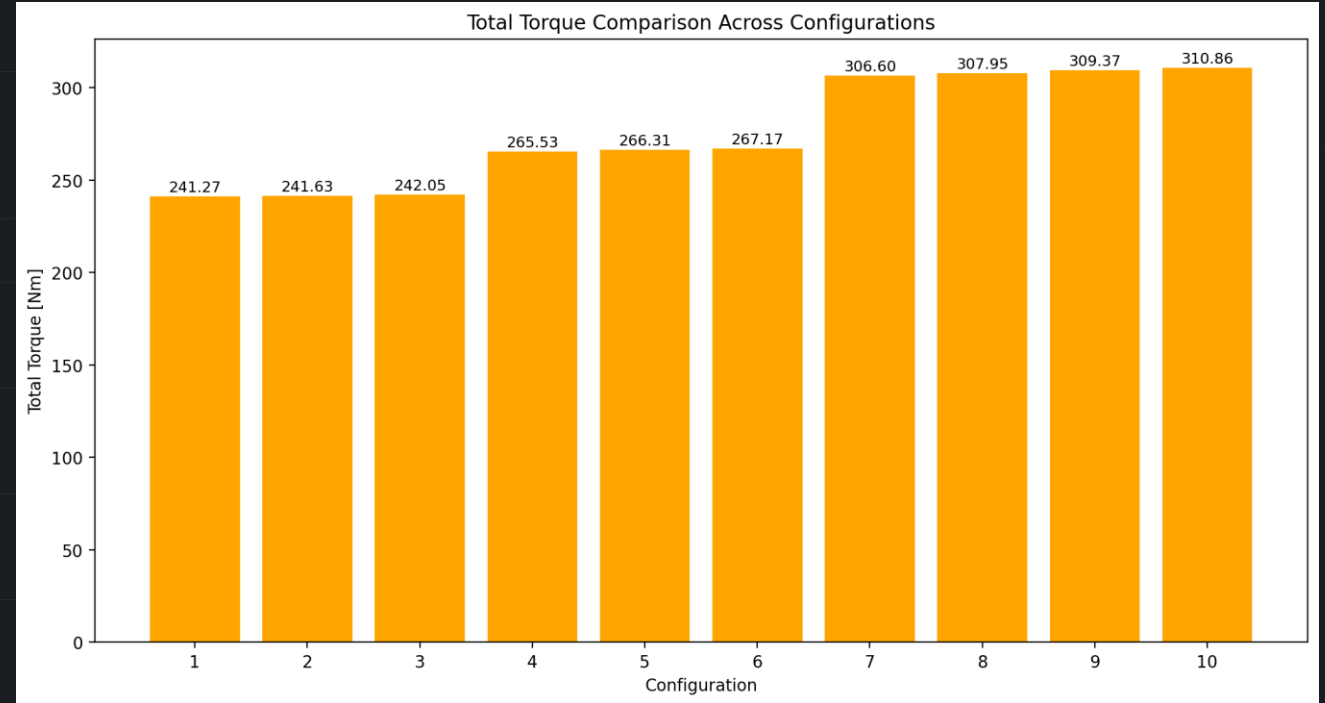
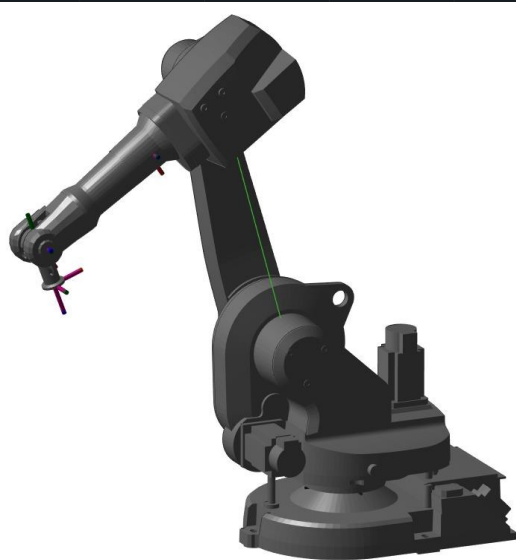
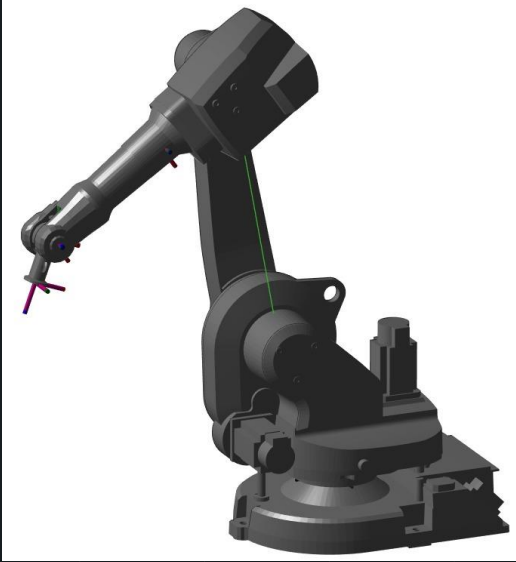
- Target EE pose TGT = [0.8, 0, 0.6];
- Position tolerance $d = 0.01$;
- Number of cycles $N = 100$;
- Iteration step $d\Theta = (\lim_qi)/N$.

Total torque per joint and configuration [Nm]

Configuration	Joint1	Joint2	Joint3	Joint4	Joint5	Joint6	TotalTorque
1	2.2748e-15	87.951	99.779	4.7471e-15	53.543	0	241.27
2	2.3149e-15	87.296	99.125	4.7325e-15	55.207	0	241.63
3	2.351e-15	86.706	98.535	4.7103e-15	56.808	0	242.05
4	5.8179e-15	105.6	96.248	4.4821e-15	63.677	0	265.53
5	5.8042e-15	105.38	96.024	4.4183e-15	64.912	0	266.31
6	5.7947e-15	105.22	95.868	4.3475e-15	66.077	0	267.17
7	7.5578e-16	142.02	93.022	3.7782e-15	71.559	0	306.6
8	7.7665e-16	142.36	93.363	3.6613e-15	72.229	0	307.95
9	8.0165e-16	142.77	93.771	3.5387e-15	72.83	0	309.37
10	8.3076e-16	143.24	94.246	3.4104e-15	73.365	0	310.86



Inverse statics



- The 10th configuration requires +65.59 Nm (28.8% more than the first one).
- Forward-shifted poses increase gravitational torque due to less favorable load distribution.

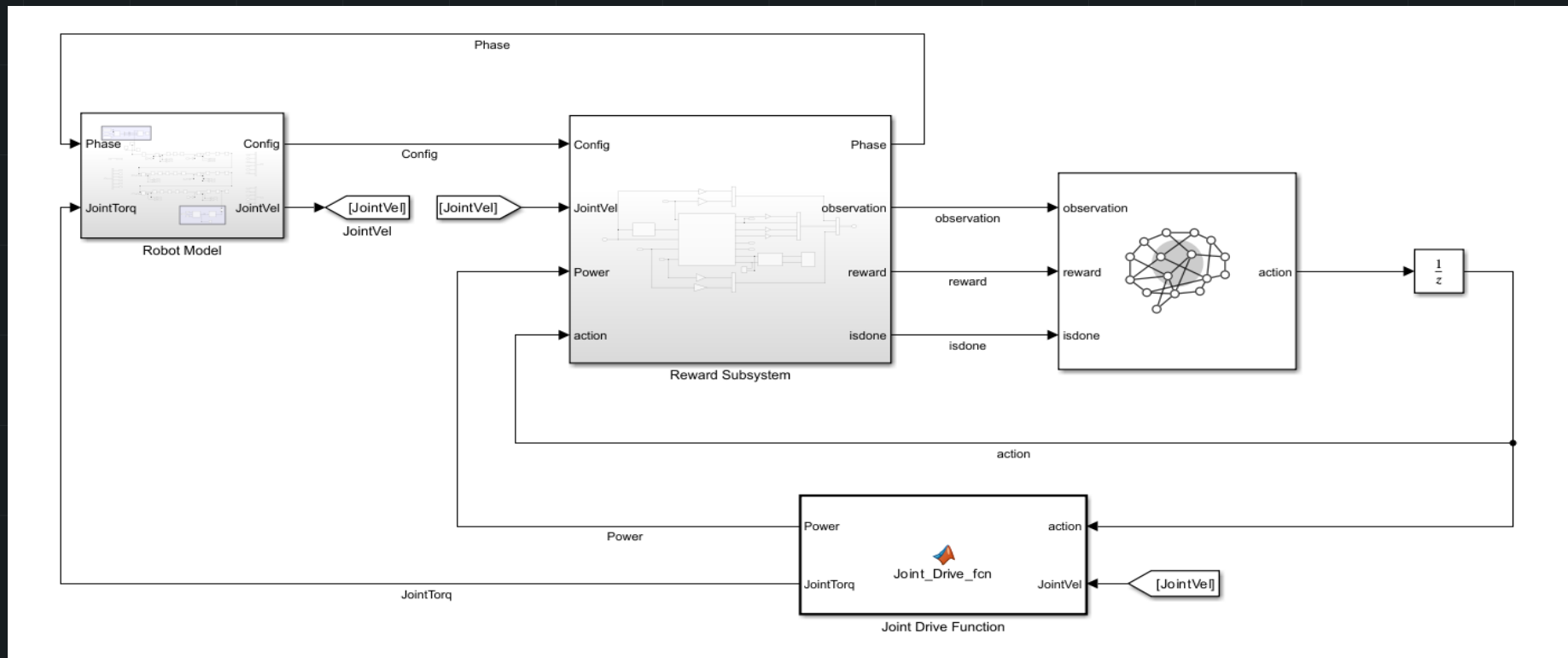


RL

**Reinforcement Learning
simulation set-up**

RL Simulation set-up

- To enable the system to learn optimal control strategies, Reinforcement Learning (RL) simulations were conducted.
- A dedicated Simulink model was developed to serve as the training environment.
- The comprehensive Simulink environment scheme integrates four primary functional blocks: the Robot Model, the Reward Subsystem, the RL Agent block, and the Joint Drive Function block.



Joint Drive Function Block

- The Joint Drive Function block acts as the actuation interface between the RL agent and the mechanical structure, computing the corresponding joint torques applied to the multibody system and the instantaneous electrical and mechanical power absorption.

$$\mathbf{i}_{\text{gear}} = [100, 100, 100, 50, 50, 50]$$
$$R_{\text{stator}} = 0.18 \, \Omega, \quad k_t = 0.45 \, \text{Nm/A}, \quad \eta_{\text{gear}} = 0.9$$

$$\tau_j = a_j \cdot i_{\text{gear},j} \cdot \eta_{\text{gear}} \quad [\text{Nm}]$$

$$P_{\text{mech},j} = a_j \cdot (\omega_j \cdot i_{\text{gear},j}) \quad [\text{W}]$$

$$P_{\text{joule},j} = R_{\text{stator}} \cdot (a_j / k_t)^2 \quad [\text{W}]$$

$$P_{\text{tot}} = \sum_j (|P_{\text{mech},j}| + P_{\text{joule},j}) \quad [\text{W}]$$

RL Agent Block

An RL TD3 agent architecture was generated via MATLAB script.

- The action space consists of a 6-dimensional vector, which dictates the control torques applied to the 6 revolute joints, symmetrically bounded within $[-100, 100]$.
- The observation space is defined by a 32-dimensional vector that comprehensively describes the state of the environment, and it is normalized within $[-1, 1]$.
- The architecture incorporates an Actor network, two distinct Critic networks, and a shared input layer, all constructed with appropriate layer dimensions.

The agent options were configured based on simulation results. The main parameters are reported in table.

	Value	Decay Rate
Target Smooth Factor	3×10^{-3}	-
Target Policy Smooth Model	0.2	1×10^{-4}
Critic Learn Rate	1×10^{-4}	-
Actor Learn Rate	5×10^{-5}	-
Exploration Model	0.8	2×10^{-4}

Reward Subsystem

- We adopted a modified reward function with respect to the reference literature.
- While the dependency on the Euclidean distance was maintained, the power consumption penalty was replaced by the norm of the joint velocities, leveraging the principle that power is proportional to the square of velocity ($P \propto V^2$).
- Furthermore, a joint configuration penalty was introduced, along with distance and configuration-dependent bonuses to refine the agent's precision.

$$R = -15 \cdot ||\Delta p||^2 - 0.1 \cdot ||\Delta q||^2 - 5 \times 10^{-4} \cdot ||\dot{q}||^2 + e^{(-50 \cdot ||\Delta p||^2)}$$

$$\text{Bonus}_{q(5)} = 0.3$$

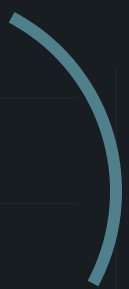
$$0 \leq q(5) \leq 1.7$$

$$\text{Bonus}_{q(1)} = 0.2$$

$$-0.05 \leq q(1) \leq 0.05$$

$$\text{Bonus}_{\Delta p} = 1.5 \cdot e^{(1 \times 10^{-3} \cdot ||\dot{q}||^2)}$$

$$\Delta p \leq 0.1$$

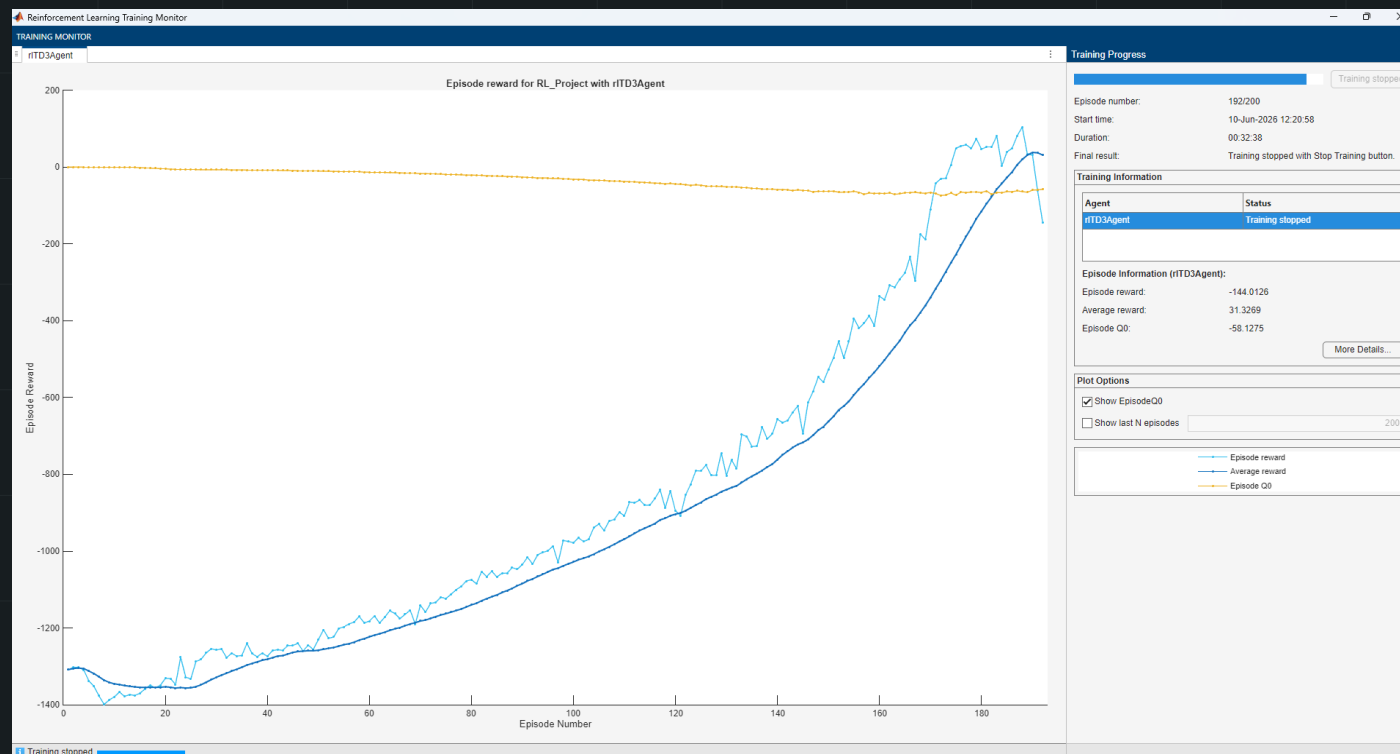


RESULTS

Training Results

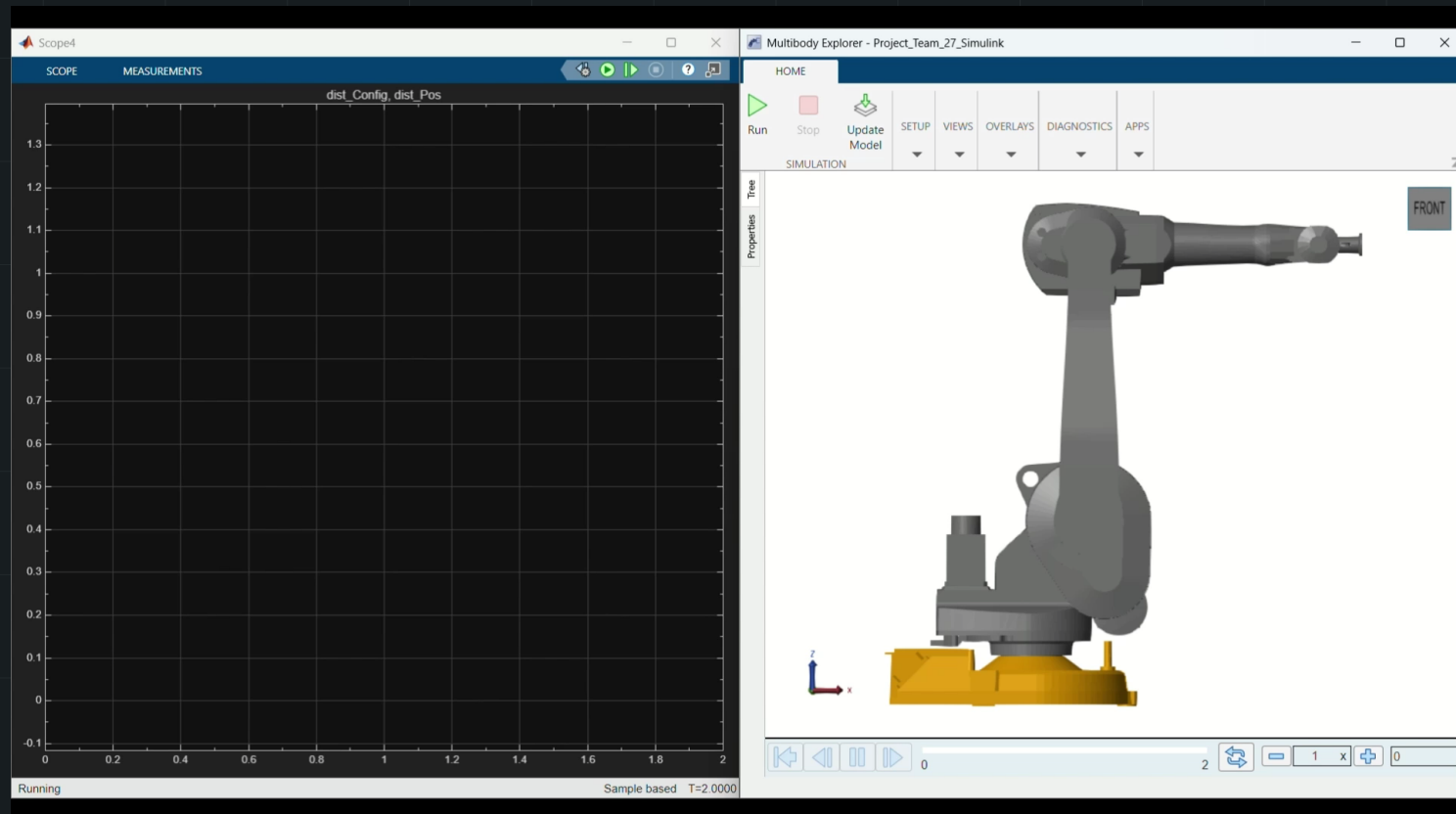
Training Results

- Finally, the training simulation for the pick phase was performed. The agent was trained for nearly 200 episodes, with an episode duration of 2 seconds.
- To prevent policy degradation and catastrophic forgetting, the training was stopped once the agent achieved convergence, demonstrating a stable learning curve and a consistent capability to reach the target position.



Training Results

- The agent demonstrated a clear learning trend, the final performance highlights some limitations, such as joint oscillations and a residual error relative to the target configuration.
- Nevertheless, the results obtained within this framework are highly satisfactory.





CONCLUSION

**Conclusion and further
implementation**

Conclusions

- Successfully combined inverse kinematics, inverse statics, and TD3 reinforcement learning within a Simulink environment.
- The proposed implementation strategy effectively optimized the control loop, yielding reasonable and physically consistent results for the initial phase.

Future Work:

- **Phase 1 Refinement:** Implement a second training session focused on the picking phase, specifically aimed at resolving the residual joint oscillations and stability issues.
- **Phase 2 Expansion:** Proceed with the placing sequence training, leveraging the model's native design to support the entire, multi-stage pick-and-place operation.
- The repository is available at the following link: <https://github.com/piazzagoffredo-ai/ABB-IRB-1600-Robotics-Project.git>